

RHCE模拟练习题

考试须知：

在考试期间，除了您所在位置的台式机外，还将使用多个虚拟系统来完成考试，但是您不具备台式机系统的root用户访问权限，但具有对虚拟系统的完全root用户访问权限。所有的考题都必须在虚拟考试系统上完成。

考试虚拟机的操作:使用桌面上的Activities --> VM Control打开虚拟系统控制台

您可以选择所要操作的虚拟机，对其进行开机（start）、关机（shutdown）、重启（reboot）断电或强制关机（poweroff）、打开显示器（console）、重置（rebuild）操作

使用桌面上的Activities --> View Exam 可以重新查看考题

在考试期间，您将操作下列虚拟系统：

system	IP Address	Role
workstation.lab.example.com	172.25.250.9	Ansible control node
servera.lab.example.com	172.25.250.10	Ansible managed node
serverb.lab.example.com	172.25.250.11	Ansible managed node
serverc.lab.example.com	172.25.250.12	Ansible managed node
serverd.lab.example.com	172.25.250.13	Ansible managed node
bastion.lab.example.com	172.25.250.254	Ansible managed node

这些系统的 IP 地址采用静态设置。请勿更改这些设置。

主机名称解析已配置为解析上方列出的完全限定主机名，同时也解析主机短名称。

帐户信息

所有系统的 root 密码是 redhat

请勿更改 root 密码。除非另有指定，否则这将是用于访问其他系统和服务的密码。此外，除非另有指定，否则此密码也应用于您创建的所有帐户，或者任何需要设置密码的服务。

为方便起见，所有系统上已预装了 SSH 密钥，允许在不输入密码的前提下通过 SSH 进行 root 访问。请勿对系统上的 root SSH 配置文件进行任何修改。

Ansible 控制节点上已创建了用户帐户 devops。此帐户预装了 SSH 密钥，允许在 Ansible 控制节点和各个 Ansible 受管节点之间进行 SSH 登录。请勿对系统上的 devops SSH 配置文件进行任何修改。您可以从 root 帐户使用 su 访问此用户帐户。

重要信息

除非另有指定，否则您的所有工作（包括 Ansible playbook、配置文件和主机清单等）应当保存在控制节点上的目录 /home/devops/ansible 中，并且应当归 devops 用户所有。所有 Ansible 相关的命令应当由 devops 用户从 Ansible 控制节点上的这个目录运行。

其他信息

一些考试项目可能需要修改 Ansible 主机清单。您要负责确保所有以前的清单组和项目保留下来，与任何其他更改共存。您还要有确保清单中所有默认的组和主机保留您进行的任何更改。

考试系统上的防火墙默认为不启用，SELinux 则处于强制模式。

如果需要安装其他软件，您的物理系统和 Ansible 控制节点可能已设置为指向 `content` 上的下述存储库：

- http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
- http://content.example.com/rhel9.0/x86_64/dvd/AppStream

一些项目需要额外的文件，这些文件已在以下位置提供：

- <http://content.example.com>

镜像注册服务器信息：

utility.lab.example.com

用户名：admin 密码：redhat

其他资源也进行了配置，供您在考试期间使用。关于这些资源的具体信息将在需要这些资源的项目中提供。

重要信息

请注意，在评分之前，您的 Ansible 受管节点系统将重置为考试开始时的初始状态，您编写的 Ansible playbook 将通过以 `devops` 用户身份从控制节点上的目录 `/home/devops/ansible` 目录运行来应用。在 `playbook` 运行后，系统会对您的受管节点进行评估，以判断它们是否按照规定进行了配置。

准备工作

```
# 准备工作，考试已经做好了。
[kiosk@foundation0 ~]$ ssh student@workstation
[student@workstation ~]$ lab start intro-install #初始化ansible导航器环境
[student@workstation ~]$ exit
[kiosk@foundation0 ~]$ ssh root@workstation
#为devops用户配置免密私钥
[root@workstation ~]$ cp /root/.ssh/lab_rsa /home/devops/.ssh/id_rsa
[root@workstation ~]$ chown devops:devops /home/devops/.ssh/id_rsa
[root@workstation ~]$ chmod 600 /home/devops/.ssh/id_rsa
[root@workstation ~]$ su - devops
[devops@workstation ~]$ ssh devops@servera #免密测试

# 教室环境bastion没有内置的devops用户，在bastion上执行创建devops用户，并设置密码和提权操作，其他主机已经做好了提权，无需操作
[root@workstation ~]$ ssh root@bastion
[root@bastion ~]$ useradd devops
[root@bastion ~]$ echo redhat | passwd --stdin devops
[root@bastion ~]$ vim /etc/sudoers.d/devops
devops ALL=(ALL) NOPASSWD: ALL
[root@bastion ~]$ exit
[root@workstation ~]$ ssh devops@bastion #免密登陆成功

# 清理教室环境中的所有被管理节点YUM源
```

```
[root@workstation ~]# for i in server{a..d} bastion;do ssh root@$i 'rm -rf
/etc/yum.repos.d/*.repo';done

# 为devops用户配置导航器容器镜像
[root@workstation ~]# sudo -u devops vim /home/devops/.ansible-navigator.yml
ansible-navigator:
  execution-environment: # 自动化执行环境
    image: utility.lab.example.com/ee-supported-rhel8:latest
    pull:
      policy: missing
  #注释
  #image 指定了使用的容器镜像, 即 utility.lab.example.com/ee-supported-
rhel8:latest。
  #pull.policy 指定了拉取策略, 这里的 missing 表示如果本地没有该镜像, 则拉取该镜像。

# 为podman配置容器镜像仓库
[root@workstation ~]# vim /etc/containers/registries.conf
[registries.search]
registries = ['utility.lab.example.com']
[registries.insecure]
registries = ['utility.lab.example.com']
```

一、安装和配置Ansible

按照下方所述, 在控制节点workstation.lab.example.com 上安装和配置 Ansible:

1. 安装所需的软件包
2. 创建名为/home/devops/ansible/inventory的静态清单文件, 以满足以下需求:
 - servera是dev主机组的成员
 - serverb是test主机组的成员
 - serverc和serverd是prod主机组的成员
 - bastion是balancers主机组的成员
 - prod组是webserver主机组的成员
3. 创建名为/home/devops/ansible/ansible.cfg的配置文件, 以满足以下要求:
 - 主机清单文件为/home/devops/ansible/inventory
 - playbook中使用的角色的位置包括/home/devops/ansible/roles
 - playbook中使用的集合的位置包括/home/devops/ansible/mycollections

```
# 实际操作
[root@workstation ~]# yum install ansible-core ansible-navigator -y
#如果考试已经安装好了, 则不需要安装了

[root@workstation ~]# su - devops #考试要求所有的配置都用一个普通用户进行配置
[devops@workstation ~]$ mkdir ansible
[devops@workstation ~]$ cd ansible/
[devops@workstation ansible]$ vim inventory
[dev]
servera
[test]
serverb
[prod]
serverc
serverd
```

```
[balancers]
bastion
[webserver:children]
prod

[devops@workstation ansible]$ ansible-config init > /tmp/ansible.cfg #导出默认配置
参考
[devops@workstation ansible]$ vim ansible.cfg
[defaults]
inventory = /home/devops/ansible/inventory
remote_user = devops
roles_path = /home/devops/ansible/roles
host_key_checking=false
collections_path = /home/devops/ansible/mycollections
[privilege_escalation]
become=True
become_method=sudo
become_user=root
become_ask_pass=False

#验证:
[devops@workstation ansible]$ ansible all -m ping
[devops@workstation ansible]$ podman login utility.lab.example.com
[devops@workstation ansible]$ ansible-navigator images #获取ansible导航器镜像
```

二、创建受管节点存储库

请按照下方所述, 创建一个名为/home/devops/ansible/yum_repo.yml的playbook, 该剧本在各个受管节点上安装yum存储库:

存储库1:

- 存储库的名称为 rh294_BASE
- 描述为 rh294 base software
- 基础URL为 http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
- GPG签名检查为启用状态
- GPG密钥URL为 http://content.example.com/rhel9.0/x86_64/dvd/RPM-GPG-KEY-redhat-releases
- 存储库为开启状态

存储库2:

- 存储库的名称为 rh294_STREAM
- 描述为 rh294 stream software
- 基础URL为 http://content.example.com/rhel9.0/x86_64/dvd/AppStream
- 签名检查为启用状态
- GPG密钥URL为 http://content.example.com/rhel9.0/x86_64/dvd/RPM-GPG-KEY-redhat-releases
- 存储库为开启状态

```
# 实际操作
[devops@workstation ansible]$ vim /home/devops/ansible/yum_repo.yml
- hosts: all
  tasks:
    - name: configure BaseOS repo
      ansible.builtin.yum_repository:
        name: rh294_BASE
```

```

        description: "rh294 base software"
        baseUrl:
http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
        enabled: yes
        gpgcheck: yes
        gpgkey: http://content.example.com/rhel9.0/x86_64/dvd/RPM-
GPG-KEY-redhat-release
        file: rhel_dvd
    - name: configure AppStream repo
      ansible.builtin.yum_repository:
        name: rh294_STREAM
        description: "rh294 stream software"
        baseUrl:
http://content.example.com/rhel9.0/x86_64/dvd/AppStream
        enabled: yes
        gpgcheck: yes
        gpgkey: http://content.example.com/rhel9.0/x86_64/dvd/RPM-
GPG-KEY-redhat-release
        file: rhel_dvd

#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run yum_repo.yml -i inventory -
m stdout
[devops@workstation ansible]$ ansible-playbook yum_repo.yml
#效果验证:
[devops@workstation ansible]$ ansible all -m shell -a 'cat
/etc/yum.repos.d/rhel_dvd.repo'
[devops@workstation ansible]$ ansible all -m shell -a 'yum clean all && yum
makecache'
```

三、安装软件包

创建一个名为 /home/devops/ansible/packages.yml 的 playbook:

1. 将 php 和 mariadb 软件包安装到 dev、test 和 prod 主机组中的主机上
2. 将 Development Tools 软件包组安装到 dev 主机组中的主机上
3. 将 dev 主机组中主机上的所有软件包更新为最新版本

```

[devops@workstation ansible]$ vim /home/devops/ansible/packages.yml
- hosts: dev, test, prod
  tasks:
    - name: install mariadb php
      ansible.builtin.yum:
        name: "{{ item }}"
        state: present
      loop:
        - php
        - mariadb

- hosts: dev
  tasks:
    - name: install Development Tools
      ansible.builtin.yum:
        name: "@Development Tools"
        state: present

- hosts: dev
```

```
tasks:
  - name: update pkgs
    ansible.builtin.yum:
      name: '*'
      state: latest
```

以下选择一种方式执行即可

```
[devops@workstation ansible]$ ansible-playbook packages.yml
```

```
[devops@workstation ansible]$ ansible-navigator run packages.yml -m stdout
```

#验证

```
[devops@workstation ansible]$ ansible dev, test, prod -m shell -a 'rpm -q php'
```

四、使用 Timesync RHEL系统角色

安装 RHEL 系统角色软件包，并创建符合以下条件的playbook

/home/devops/ansible/timesync.yml:

1. 在所有受管节点上运行
2. 使用 timesync 角色
3. 配置该角色，以使用当前有效的 NTP 提供
4. 配置该角色，以使用时间服务器 classroom.example.com

```
[root@workstation ~]# yum -y install rhel-system-roles
```

```
[root@workstation ~]# su - devops
```

```
[devops@workstation ~]$ cd ansible/
```

```
[devops@workstation ansible]$ cp -r /usr/share/ansible/roles/rhel-system-roles.timesync/ roles/timesync
```

#查帮助

```
[devops@workstation ansible]$ vim roles/timesync/README.md
```

```
```yam
```

```
- hosts: targets
```

```
vars:
```

```
 timesync_ntp_servers:
```

```
 - hostname: foo.example.com
```

```
 iburst: yes
```

```
 - hostname: bar.example.com
```

```
 iburst: yes
```

```
 - hostname: baz.example.com
```

```
 iburst: yes
```

```
roles:
```

```
 - rhel-system-roles.timesync
```

# 编写yam文件

```
[devops@workstation ansible]$ vim timesync.yml
```

```
- hosts: all
```

```
vars:
```

```
 timesync_ntp_servers:
```

```
 - hostname: classroom.example.com
```

```
 iburst: yes
```

```
roles:
```

```
 - timesync
```

# 以下选择一种方式执行即可

```
[devops@workstation ansible]$ ansible-playbook timesync.yml
```

```
[devops@workstation ansible]$ ansible-navigator run timesync.yml -m stdout
```

#验证

```
[devops@workstation ansible]$ ansible all -m shell -a 'chronyc sources
```

## 五、使用 SELINUX RHEL系统角色

安装RHEL角色，并使用seLinux角色，要求在所有节点运行，将SELINUX设置为强制模式

```
[devops@workstation ansible]$ sudo yum -y install rhel-system-roles
[sudo] password for devops: devops
[devops@workstation ansible]$ mkdir roles
[devops@workstation ansible]$ cd roles/
[devops@workstation ansible]$ cp -r /usr/share/ansible/roles/rhel-system-roles.selinux/ roles/selinux
[devops@workstation ansible]$ vim roles/selinux/README.md #参考配置
```yam1
vars:
  [ see below ]
roles:
  - role: rhel-system-roles.selinux
    become: true
```
```yam1
selinux_policy: targeted
selinux_state: enforcing
```

[devops@workstation ansible]$ vim selinux.yml #看题目要求的文件名是什么
- hosts: all
 vars:
 selinux_policy: targeted
 selinux_state: enforcing
 roles:
 - role: selinux
 become: true

#修改任意一台节点的slinux状态，执行playbook后验证（考试时无需操作）
[devops@workstation ansible]$ ssh root@servera 'setenforce 0'
以下选择一种方式执行即可
[devops@workstation ansible]$ ansible-playbook selinux.yml
[devops@workstation ansible]$ ansible-navigator run selinux.yml -m stdout
#验证
[devops@workstation ansible]$ ansible all -m shell -a 'getenforce'
```

## 六、使用Ansible Galaxy安装角色

使用 Ansible Galaxy 和要求文件 /home/devops/ansible/roles/requirements.yml，从以下 URL 下载角色并安装到 /home/devops/ansible/roles:

- <http://classroom.example.com/content/haproxy.tar.gz> 此角色的名称应当为 balancer
- <http://classroom.example.com/content/phpinfo.tar.gz> 此角色的名称应当为 phpinfo

# 准备工作：将haproxy.tar.gz和phpinfo.tar.gz下载下来放入foundation0下面的/content目录下。

# 实际操作

```
[devops@workstation ansible]$ vim roles/requirements.yml
- name: balancer
 src: http://classroom.example.com/content/haproxy.tar.gz
- name: phpinfo
 src: http://classroom.example.com/content/phpinfo.tar.gz

[devops@workstation ansible]$ ansible-galaxy install -r roles/requirements.yml -p roles/
```

## 七、安装集合

将<http://content.example.com/>以下集合，下载到/home/devops/ansible/mycollections目录下，并面向用户devops

ansible-posix-1.5.1.tar.gz

community-general-6.3.0.tar.gz

#实际操作

```
[devops@workstation ansible]$ ansible-galaxy collection install
http://content.example.com/ansible-posix-1.5.1.tar.gz -p mycollections/
[devops@workstation ansible]$ ansible-galaxy collection install
http://content.example.com/community-general-6.3.0.tar.gz -p mycollections/
#验证
[devops@workstation ansible]$ ls mycollections/ansible_collections/
```

## 八、创建和使用角色

根据下列要求，在 /home/devops/ansible/roles中创建名为 apache 的角色：

1. httpd软件包已安装，设为在系统启动时启用
2. 防火墙已启用并正在运行，并使用允许访问 Web 服务器的规则
3. 模板文件 index.html.j2 已存在，用于创建具有以下输出的文件 /var/www/html/index.html:
  - Welcome to HOSTNAME on IPADDRESS
  - 其中，HOSTNAME 是受管节点的完全限定域名，IPADDRESS 则是受管节点的 IP 地址。
4. 按照下方所述，创建一个使用此角色的 playbook /home/devops/ansible/newrole.yml:
  - 该 playbook 在 webserver 主机组中的主机上运行

```
[devops@workstation ansible]$ cd roles/
[devops@workstation roles]$ ansible-galaxy init apache
[devops@workstation roles]$ vim apache/tasks/main.yml

tasks file for apache
- name: install http
 yum:
 name: httpd
 state: present
- name: config system service
```



```

service:
 name: "{{ item }}"
 state: started
 enabled: yes
loop:
 - httpd
 - firewalld
- name: firewalld service
 firewalld:
 service: http
 permanent: yes
 immediate: yes
 state: enabled
- name: user templates
 template:
 src: index.html.j2
 dest: /var/www/html/index.html
[devops@workstation roles]$ vim apache/templates/index.html.j2
welcome to {{ ansible_fqdn }} on {{ ansible_default_ipv4.address }}
[devops@workstation roles]$ cd ..
[devops@workstation ansible]$
[devops@workstation ansible]$ vim newrole.yml
- name: use apache role
 hosts: webserver
 roles:
 - apache
#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run newrole.yml -m stdout
[devops@workstation ansible]$ ansible-playbook newrole.yml
#验证
[devops@workstation ansible]$ curl serverc
welcome to serverc.lab.example.com on 172.25.250.12
[devops@workstation ansible]$ curl serverd
welcome to serverd.lab.example.com on 172.25.250.13

```

## 九、从Ansible Galaxy使用角色

根据下列要求，创建一个名为 /home/devops/ansible/roles.yml 的 playbook：

- playbook 中包含一个 play，该 play 在 balancers 主机组中的主机上运行并将使用 balancer 角色。
  - 此角色配置一项服务，以在 webserver 主机组中的主机之间平衡 Web 服务器请求的负载。
  - 浏览到 balancers 主机组中的主机（例如<http://bastion.lab.example.com/>）将生成以下输出：
    - Welcome to serverc.example.com on 172.25.250.12
  - 重新加载浏览器将从另一 Web 服务器生成输出：
    - Welcome to serverd.example.com on 172.25.250.13
- playbook 中包含一个 play，该 play 在 webserver 主机组中的主机上运行并将使用 phpinfo 角色。
  - 通过 URL /hello.php 浏览到 webserver 主机组中的主机将生成以下输出：
    - Hello PHP World from FQDN
    - 其中，FQDN是主机的完全限定名称。
  - 例如，浏览到 <http://serverc.lab.example.com/hello.php> 会生成以下输出：

- Hello PHP World from serverc.lab.example.com
- 另外还有 PHP 配置的各种详细信息，如安装的PHP 版本等。
- 同样，浏览到 <http://serverd.lab.example.com/hello.php> 会生成以下输出：
  - Hello PHP World from serverd.lab.example.com
  - 另外还有 PHP 配置的各种详细信息，如安装的PHP 版本等。

```
准备工作，关闭bastion主机上的httpd服务，以免冲突，考试不需要做
[devops@workstation ansible]$ ssh root@bastion 'systemctl stop httpd & systemctl
disable httpd'

实际操作
[devops@workstation ansible]$ vim roles.yml
- hosts: webservers

- hosts: balancers
 roles:
 - balancer

- hosts: webservers
 roles:
 - phpinfo

#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run roles.yml -m stdout
[devops@workstation ansible]$ ansible-playbook roles.yml

验证
[devops@workstation ansible]$ curl http://bastion.lab.example.com/
welcome to serverc.lab.example.com on 172.25.250.12
[devops@workstation ansible]$ curl http://bastion.lab.example.com/
welcome to serverd.lab.example.com on 172.25.250.13
[devops@workstation ansible]$ curl http://serverc.lab.example.com/hello.php
Hello PHP World form serverc.lab.example.com
[devops@workstation ansible]$ curl http://serverd.lab.example.com/hello.php
Hello PHP World form serverd.lab.example.com
```

## 十、创建和使用逻辑卷

创建一个名为/home/devops/ansible/lv.yml 的playbook，它将在所有受管节点上运行以执行下列任务

1. 创建符合以下要求的逻辑卷：
  - 逻辑卷创建在 research 卷组中
  - 逻辑卷名称为 data
  - 逻辑卷大小为 600MiB
2. 使用 ext4 文件系统格式化逻辑卷
3. 如果无法创建请求的逻辑卷大小，应显示错误消息
  - Could not create logical volume of that size, 并且应改为使用大小 400MiB。
4. 如果卷组research 不存在，应显示错误消息
  - Volume group does not exist.
5. 不要以任何方式挂载逻辑卷。

```
准备工作，5台虚拟机均没有对应的卷组和逻辑卷，因此先准备分区和卷组(考试不需要)，其中设置
server[a:b] 卷组为800M,server[c:d] 卷组为500M, bastion没有卷组
```

```
[devops@workstation ansible]$ vim lvm_pre.yml
- hosts: dev,test
 tasks:
 - name: Create partition for LVM
 parted:
 device: /dev/vdb
 number: 1
 flags: [lvm]
 state: present
 part_start: 1MiB
 part_end: 800MiB
 - name: create research vg
 lvg:
 vg: research
 pvs: /dev/vdb1

- hosts: prod
 tasks:
 - name: Create partition for LVM
 parted:
 device: /dev/vdb
 number: 1
 flags: [lvm]
 state: present
 part_start: 1MiB
 part_end: 600MiB
 - name: create research vg
 lvg:
 vg: research
 pvs: /dev/vdb1

[devops@workstation ansible]$ ansible-playbook lvm_pre.yml
```

#### # 验证

```
[devops@workstation ansible]$ ssh root@servera vgs
VG #PV #LV #SN Attr VSize VFree
research 1 0 0 wz--n- 796.00m 796.00m
[devops@workstation ansible]$ ssh root@serverb vgs
VG #PV #LV #SN Attr VSize VFree
research 1 0 0 wz--n- 796.00m 796.00m
[devops@workstation ansible]$ ssh root@serverc vgs
VG #PV #LV #SN Attr VSize VFree
research 1 0 0 wz--n- 596.00m 596.00m
[devops@workstation ansible]$ ssh root@serverd vgs
VG #PV #LV #SN Attr VSize VFree
research 1 0 0 wz--n- 596.00m 596.00m
[devops@workstation ansible]$ ssh root@bastion vgs
```

#### # 实际操作，考试只需要做以下步骤即可

```
[devops@workstation ansible]$ vim lv.yml
- hosts: all
 tasks:
 - block:
 - name: create lvm 600m
 lvol:
 vg: research
 lv: data
```

```

 size: 600m
 rescue:
 - name: output fail msg
 debug:
 msg: Could not create logical volume of that size
 - name: create lvm 400m
 lvol:
 vg: research
 lv: data
 size: 400m
 always:
 - name: format lvm
 filesystem:
 fstype: ext4
 dev: /dev/research/data
 when: "'research' in ansible_lvm.vgs"
 - name: search not exists
 debug:
 msg: Volume group does not exist
 when: "'research' not in ansible_lvm.vgs"
#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run lv.yml -m stdout
[devops@workstation ansible]$ ansible-playbook lv.yml

验证
[devops@workstation ~]$ ssh root@servera lvs
LV VG Attr LSize Pool Origin Data% Meta% Move Log Cpy%Sync Convert
data research -wi-a----- 600.00m
[devops@workstation ~]$ ssh root@serverb lvs
LV VG Attr LSize Pool Origin Data% Meta% Move Log Cpy%Sync Convert
data research -wi-a----- 600.00m
[devops@workstation ~]$ ssh root@serverc lvs

LV VG Attr LSize Pool Origin Data% Meta% Move Log Cpy%Sync Convert
data research -wi-a----- 400.00m
[devops@workstation ~]$ ssh root@serverd lvs
LV VG Attr LSize Pool Origin Data% Meta% Move Log Cpy%Sync Convert
data research -wi-a----- 400.00m

```

## 十一、创建分区

说明：由于只有一块可用磁盘，与上一道题冲突，需要重置磁盘后再做

创建一个名为/home/devops/ansible/parted.yml的playbook，它将在dev主机组上运行下列任务

1. 如果磁盘/dev/vdd存在，则创建1500m分区
2. 如果无法创建请求的分区大小，应显示错误消息
  - Could not create partition of that size，并且应改为使用大小 800m。
3. 如果磁盘/dev/vdd不存在，应显示错误消息
  - disk /dev/vdd does not exist。
4. 如果磁盘/dev/vdb存在，则创建1500m分区
5. 如果无法创建请求的分区大小，应显示错误消息

- Could not create partition of that size, 并且应改为使用大小 800m。
6. 最后分区都要格式化为ext4文件系统, 并挂载在/mnt/fs01上

#重置磁盘 正式考试不需要做

```
[devops@workstation ansible]$ ansible all -m shell -a 'lvremove /dev/research/data -y'
[devops@workstation ansible]$ ansible all -m shell -a 'vgremove research -y'
[devops@workstation ansible]$ ansible all -m shell -a 'pvremove /dev/vdb1'
[devops@workstation ansible]$ ansible all -m shell -a 'dd if=/dev/zero of=/dev/vdb bs=512 count=1'
#检查是否重置成功, 所有节点vdb磁盘没有分区则重置成功
[devops@workstation ansible]$ ansible all -m shell -a 'fdisk -l /dev/vdb'
```

#实际操作

```
[devops@workstation ansible]$ vim parted.yml
- hosts: dev
 tasks:
 - block:
 - name: create 1500m
 parted:
 device: /dev/vdd
 number: 1
 state: present
 part_end: 1501MiB
 rescue:
 - name: output fail msg
 debug:
 msg: Could not create partition of that size
 - name: create 800MiB
 parted:
 device: /dev/vdd
 number: 1
 state: present
 part_end: 801MiB
 always:
 - name: format partition
 filesystem:
 fstype: ext4
 dev: /dev/vdd1
 - name: mount device
 mount:
 path: /mnt/fs01
 src: /dev/vdd1
 fstype: ext4
 opts: defaults
 state: mounted
 when: "'vdd' in ansible_devices"

 - name: vdb not exists
 debug:
 msg: disk /dev/vdb does not exist
 when: "'vdb' not in ansible_devices"

 - block:
 - name: create 1500MiB
 parted:
 device: /dev/vdb
```

```

 number: 1
 state: present
 part_end: 1501MiB
 rescue:
 - name: output fail msg
 debug:
 msg: Could not create partition of that size
 - name: create 800MiB
 parted:
 device: /dev/vdb
 number: 1
 state: present
 part_end: 801MiB
 always:
 - name: format partition
 filesystem:
 fstype: ext4
 dev: /dev/vdb1
 - name: mount device
 mount:
 path: /mnt/fs01
 src: /dev/vdb1
 fstype: ext4
 opts: defaults
 state: mounted
 when: "'vdb' in ansible_devices and 'vdd' not in ansible_devices"
- name: vdb not exists
 debug:
 msg: disk /dev/vdb does not exist
 when: "'vdb' not in ansible_devices and 'vdd' not in ansible_devices"

```

#以下两种方式任选一种测试执行

```
[devops@workstation ansible]$ ansible-playbook parted.yml
```

```
[devops@workstation ansible]$ ansible-navigator run parted.yml -m stdout
```

#验证

```
[devops@workstation ansible]$ ssh root@servera lsblk
```

| NAME   | MAJ:MIN | RM | SIZE | RO | TYPE | MOUNTPOINT |
|--------|---------|----|------|----|------|------------|
| vda    | 252:0   | 0  | 10G  | 0  | disk |            |
| ├─vda1 | 252:1   | 0  | 1M   | 0  | part |            |
| ├─vda2 | 252:2   | 0  | 100M | 0  | part | /boot/efi  |
| └─vda3 | 252:3   | 0  | 9.9G | 0  | part | /          |
| vdb    | 252:16  | 0  | 1G   | 0  | disk |            |
| └─vdb1 | 252:17  | 0  | 800M | 0  | part | /mnt/fs01  |

## 十二、生成主机文件

1. 编写模板文件/home/devops/ansible/hosts.j2，针对每个清单主机包含一行内容，其格式与/etc/hosts 相同。
2. 创建名为 /home/devops/ansible/hosts.yml 的playbook，它将使用此模板在 dev 主机组中的主机上生成文件 /etc/myhosts。

3. 该 playbook 运行后，dev 主机组中主机上的文件/etc/myhosts 应针对每个受管主机包含一行内容。

```
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
172.25.254.10 servera.lab.example.com servera
172.25.254.11 serverb.lab.example.com serverb
172.25.254.12 serverc.lab.example.com serverc
172.25.254.13 serverd.lab.example.com serverd
172.25.250.254 bastion.lab.example.com bastion
```

注：清单主机名称的显示顺序不重要。

```
实际操作
[devops@workstation ansible]$ vim hosts.j2
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
{% for host in groups.all %}
{{ hostvars[host].ansible_eth0.ipv4.address }} {{ hostvars[host].ansible_fqdn }}
{{ hostvars[host].ansible_hostname }}
{% endfor %}
[devops@workstation ansible]$ vim hosts.yml
- hosts: all
- hosts: dev
 tasks:
 - name: copy hosts.j2 to dev
 template:
 src: hosts.j2
 dest: /etc/myhosts
#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run hosts.yml -m stdout
[devops@workstation ansible]$ ansible-playbook hosts.yml

验证
[devops@workstation ansible]$ ssh root@servera 'cat /etc/myhosts'
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
172.25.250.10 servera.lab.example.com servera
172.25.250.11 serverb.lab.example.com serverb
172.25.250.254 bastion.lab.example.com bastion
172.25.250.12 serverc.lab.example.com serverc
172.25.250.13 serverd.lab.example.com serverd
注：清单主机名称的显示顺序不重要。
```

## 十三、修改文件内容

按照下方所述，创建一个名为 /home/devops/ansible/issue.yml 的 playbook：

1. 该 playbook 将在所有清单主机上运行
2. 该 playbook 会将 /etc/issue 的内容替换为下方所示的一行文本：
  - 在 dev 主机组中的主机上，这行文本显示为：Development
  - 在 test 主机组中的主机上，这行文本显示为：Test
  - 在 prod 主机组中的主机上，这行文本显示为：Production

```
[devops@workstation ansible]$ vim issue.yml
```

```
- hosts: all
 tasks:
 - name: write Development to dev
 copy:
 content: "Development\n"
 dest: /etc/issue
 when: "'dev' in group_names"
 - name: write Test to test
 copy:
 content: "Test\n"
 dest: /etc/issue
 when: "'test' in group_names"
 - name: write Production to prod
 copy:
 content: "Production\n"
 dest: /etc/issue
 when: "'prod' in group_names"
```

#以下两种方式任选一种测试执行

```
[devops@workstation ansible]$ ansible-navigator run issue.yml -m stdout
[devops@workstation ansible]$ ansible-playbook issue.yml
```

# 验证

```
[devops@workstation ansible]$ ansible dev,test,prod -m shell -a 'cat /etc/issue'
serverc | CHANGED | rc=0 >>
Production
servera | CHANGED | rc=0 >>
Development
serverb | CHANGED | rc=0 >>
Test
serverd | CHANGED | rc=0 >>
Production
```

## 十四、创建Web内容目录

按照下方所述，创建一个名为 /home/devops/ansible/webcontent.yml 的 playbook：

1. 该 playbook 在 dev 主机组中的受管节点上运行
2. 创建符合下列要求的目录 /webdev:
  - 拥有组为 devops 组
  - 具有常规权限: owner=read+write+execute, group=read+write+execute, other=read+execute
  - 具有特殊权限: set group ID
3. 用符号链接将 /var/www/html/webdev 链接到 /webdev
4. 创建文件 /webdev/index.html，其中包含如下所示的单行文本：Development
5. 在 dev 主机组中主机上浏览此目录（例如 <http://servera.lab.example.com/webdev/>）将生成以下输出：
  - Development

```
[devops@workstation ansible]$ vim webcontent.yml
- hosts: dev
 tasks:
 - name: install httpd
 yum:
```



```

 name: httpd
 state: present
- name: enable httpd
 systemd:
 name: httpd
 enabled: yes
 state: started
- name: enable 80/tcp
 firewallld:
 service: http
 immediate: yes
 permanent: yes
 state: enabled

- name: Create webdev directory
 file:
 path: /webdev
 state: directory
 owner: root
 group: devops
 mode: '2775'
 setype: httpd_sys_content_t
- name: Create file
 copy:
 content: "Development\n"
 dest: /webdev/index.html
 setype: httpd_sys_content_t

- name: Create soft link
 file:
 src: /webdev
 dest: /var/www/html/webdev
 state: link

#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run webcontent.yml -m stdout
[devops@workstation ansible]$ ansible-playbook webcontent.yml

验证
[devops@workstation ansible]$ curl
http://servera.lab.example.com/webdev/index.html
Development

```

## 十五、生成硬件报告

创建一个名为 /home/devops/ansible/hwreport.yml 的 playbook，它将在所有受管节点上生成含有以下信息的输出文件 /root/hwreport.txt：

- 清单主机名称
- 以 MB 表示的总内存大小
- BIOS 版本
- 磁盘设备 vda 的大小
- 磁盘设备 vdb 的大小
- 输出文件中的每一行含有一个 key=value 对。

您的 playbook 应当：

1. 从 <http://172.25.254.254/content/hwreport.empty> 下载文件，并将它保存为 /root/hwreport.txt

2. 使用正确的值修改 /root/hwreport.txt
3. 如果硬件项不存在, 相关的值应设为 NONE

#### # 实际操作

```
[devops@workstation ansible]$ vim hwreport.yml
- hosts: all
 tasks:
 - name: Create report file
 get_url:
 url: http://172.25.254.254/content/hwreport.empty
 dest: /root/hwreport.txt
 - name: Get inventory_hostname
 replace:
 path: /root/hwreport.txt
 regexp: 'inventoryhostname'
 replace: '{{ inventory_hostname }}'
 - name: Get memory total size
 replace:
 path: /root/hwreport.txt
 regexp: 'memory_in_MB'
 replace: '{{ ansible_memtotal_mb | string }}'
 - name: Get bios version
 replace:
 path: /root/hwreport.txt
 regexp: 'BIOS_version'
 replace: '{{ ansible_bios_version }}'
 - name: Get disk vda size
 replace:
 path: /root/hwreport.txt
 regexp: 'disk_vda_size'
 replace: '{{ ansible_devices.vda.size | default('NONE') }}'
 - name: Get disk vdb size
 replace:
 path: /root/hwreport.txt
 regexp: 'disk_vdb_size'
 replace: '{{ ansible_devices.vdb.size | default('NONE') }}'
```

#### # 以下两种方式任选一种测试执行

```
[devops@workstation ansible]$ ansible-navigator run hwreport.yml -m stdout
[devops@workstation ansible]$ ansible-playbook hwreport.yml
```

#### # 验证

```
[devops@workstation ansible]$ ansible all -m shell -a 'cat /root/hwreport.txt'
bastion | CHANGED | rc=0 >>
hostname = bastion
memory = 960
bios_version = 1.15.0-1.e19
vda_size = 10.00 GB
vdb_size = NONE
serverb | CHANGED | rc=0 >>
hostname = serverb
memory = 960
bios_version = 1.15.0-1.e19
vda_size = 10.00 GB
vdb_size = 1.00 GB
```

```
servera | CHANGED | rc=0 >>
hostname = servera
memory = 960
bios_version = 1.15.0-1.e19
vda_size = 10.00 GB
vdb_size = 1.00 GB
serverd | CHANGED | rc=0 >>
hostname = serverd
memory = 960
bios_version = 1.15.0-1.e19
vda_size = 10.00 GB
vdb_size = 1.00 GB
serverc | CHANGED | rc=0 >>
hostname = serverc
memory = 960
bios_version = 1.15.0-1.e19
vda_size = 10.00 GB
vdb_size = 1.00 GB
```

## 十六、创建密码库

按照下方所述，创建一个 Ansible 库来存储用户密码：

1. 库名称为 /home/devops/ansible/locker.yml
2. 库中含有两个变量，名称如下：
  - pw\_developer，值为 Imadev
  - pw\_manager，值为 Imamgr
3. 用于加密和解密该库的密码为 fgydsxsxs bxs
4. 密码存储在文件 /home/devops/ansible/secret.txt 中

```
[devops@workstation ansible]$ vim locker.yml
pw_developer: Imadev
pw_manager: Imamgr
[devops@workstation ansible]$ echo fgydsxsxs bxs > secret.txt
[devops@workstation ansible]$ ansible-vault encrypt --vault-password-
file=secret.txt locker.yml
验证
[devops@workstation ansible]$ ansible-vault view locker.yml
Vault password: # 输入密码
pw_developer: Imadev
pw_manager: Imamgr
```

## 十七、创建用户账户

1. 从 [http://172.25.254.254/content/user\\_list.yml](http://172.25.254.254/content/user_list.yml) 下载要创建的用户列表，并将它保存到 /home/devops/ansible，用户密码来自于 /home/devops/ansible/locker.yml 文件。
2. 创建名为 /home/devops/ansible/users.yml 的 playbook，从而按以下所述创建用户帐户：
  - 职位描述为 developer 的用户应当：
    - 在 dev 和 test 主机组中的受管节点上创建
    - 从 pw\_developer 变量分配密码
    - 是附加组 devops 的成员
  - 职位描述为 manager 的用户应当：
    - 在 prod 主机组中的受管节点上创建

- 从 pw\_manager 变量分配密码
  - 是附加组 opsmgr 的成员
3. 密码应采用 SHA512 哈希格式。
4. 您的 playbook 应能够在本次考试中使用在其他位置创建的库密码文件 /home/devops/ansible/secret.txt 正常运行。

```
实际操作
[devops@workstation ansible]$ wget http://172.25.254.254/content/user_list.yml
[devops@workstation ansible]$ vim users.yml
- hosts: dev, test
 vars_files:
 - locker.yml
 - user_list.yml
 tasks:
 - name: Ensure group "devops" exists
 group:
 name: devops
 state: present
 - name: Create user in developer
 user:
 name: "{{ item.name }}"
 uid: "{{ item.uid }}"
 groups: devops
 password: "{{ pw_developer | password_hash('sha512') }}"
 loop: "{{ users }}"
 when: item.job == 'developer'

- hosts: prod
 vars_files:
 - locker.yml
 - user_list.yml
 tasks:
 - name: Ensure group "opsmgr" exists
 group:
 name: opsmgr
 state: present
 - name: Create user in manager
 user:
 name: "{{ item.name }}"
 uid: "{{ item.uid }}"
 groups: opsmgr
 password: "{{ pw_manager | password_hash('sha512') }}"
 loop: "{{ users }}"
 when: item.job == 'manager'

#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run users.yml -m stdout --
vault-password-file=secret.txt
[devops@workstation ansible]$ ansible-playbook --vault-password-file=secret.txt
users.yml
```

## 十八、更新Ansible库的密钥

按照下方所述，更新现有 Ansible 库的密钥：

1. 从 <http://172.25.254.254/content/salaries.yml> 下载 Ansible 库到 /home/devops/ansible
2. 当前的库密码为 insecure4sure
3. 新的库密码为 bbe2de98389b
4. 库使用新密码保持加密状态

# 实际操作

```
[devops@workstation ansible]$ wget http://172.25.254.254/content/salaries.yml
[devops@workstation ansible]$ ansible-vault rekey salaries.yml
Vault password: # 输入旧密码insecure4sure
New Vault password: # 输入新密码bbe2de98389b
Confirm New Vault password: # 输入新密码bbe2de98389b
Rekey successful
```

## 十九、创建计划任务

为natasha创建一个计划任务，要求每隔2分钟执行一次echo hello，playbook文件名为cron.yml，该playbook再dev主机组上运行

```
[devops@workstation ansible]$ vim cron.yml
- hosts: dev
 tasks:
 - name: create natasha #该用户如果已经创建，此任务可以忽略
 user:
 name: natasha
 state: present
 - name: create cron tasks
 cron:
 name: "exec tasks every 2 minute"
 minute: "*/2"
 user: natasha
 job: "echo hello" #复制题目给的任务即可

#以下两种方式任选一种测试执行
[devops@workstation ansible]$ ansible-navigator run cron.yml -m stdout
[devops@workstation ansible]$ ansible-playbook cron.yml

#验证
[root@servera ~]# crontab -l -u natasha
#Ansible: exec tasks every 2 minute
*/2 * * * * echo hello
```